Practical 1

Step1

#include <stdio.h> float avg(){ int sum; for (int i = 0; i = n; i++){ sum += arr[i]; } retrurn sum/n; } int main() { // Write C code here int data[5] = {10, 20, 30, 40, 51}: printf("average =%f\n", (average data , 5)); return 0; } fix my code

Yes. Your code has several syntax and logic errors. Here is the **fully corrected version**:

C

```c
#include <stdio.h>

float avg(int arr[], int n) {
    int sum = 0;

    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }

    return (float)sum / n;
}

int main() {
    int data[5] = {10, 20, 30, 40, 51};

    printf("average = %f\n", avg(data, 5));

    return 0;
}
```

Output

```
average = 30.200001
```

Ask anything Think

Step 2

you are a senior c engineer reviewing code written by a 2nd year student the program should calculate an integer array average and print a decimal . list every bugs do not fix them. numbered list with line number and one line description. #include <stdio.h>
float avg(){

```
int sum;
for (int i = 0; i = n; i++){
   sum += arr[i];
}
retrurn sum/n;
```

Show more

Bugs

1. **Line 2** — `avg()` uses `n` and `arr`, but neither variable is declared or passed to the function.
2. **Line 4** — `sum` is uninitialized, so `sum += arr[i]` uses an indeterminate value.
3. **Line 5** — `i = n` assigns `n` to `i` instead of comparing `i` with `n`, causing incorrect loop behavior.
4. **Line 5** — `n` is undeclared, so the program cannot compile.
5. **Line 6** — `retrurn` is misspelled; `return` is required.
6. **Line 6** — `arr` and `n` are undeclared in `avg()`, so `sum/n` cannot be evaluated.
7. **Line 10** — The array declaration ends with `:` instead of `;`, causing a syntax error.
8. **Line 11** — `average` is not a declared function; the defined function is named `avg`.
9. **Line 11** — `data` is passed without the syntax required for a function argument list.
10. **Line 11** — The `avg` function is not actually called, so the array average is never calculated.
11. **Line 11** — The `printf` argument is invalid because `(average data, 5)` is not valid C function-call syntax.
12. **Line 11** — The `%f` format expects a `double`, but the expression supplied is not a valid floating-point average.

Ask anything Think

Step 3

for each bug you listed ,explain in 2 3 lines what the program actually does at runtime because of it and what it was suppoesed to do . donot give corrected code yet

Sure. **No corrected code yet.** Since several bugs are compile-time errors, technically the program **never reaches runtime** until those are fixed.

Bug-by-bug runtime/expected behavior

1. **Line 2 — `arr` and `n` are undeclared**
   Actual: The compiler reports that `arr` and `n` are undeclared, so the program fails to compile.
   Supposed: `avg()` should receive the array and its size so it can calculate the average.
2. **Line 4 — `sum` is uninitialized**
   Actual: If compilation were possible, `sum` would start with an indeterminate value, producing an unpredictable average.
   Supposed: `sum` should start from zero before adding array elements.
3. **Line 5 — `i = n` instead of `i < n`**
   Actual: The statement assigns `n` to `i` instead of checking whether `i` is less than `n`, causing incorrect loop control.
   Supposed: The loop should visit every array element from index `0` through `n-1`.
4. **Line 5 — `n` is undeclared**
   Actual: The compiler cannot determine what `n` means and stops compilation.
   Supposed: `n` should represent the number of elements in the array.
5. **Line 6 — `retrurn` is misspelled**
   Actual: The compiler treats `retrurn` as an unknown identifier/invalid statement, so compilation fails.
   Supposed: The function should return the calculated floating-point average.
6. **Line 6 — `arr` and `n` are undeclared**
   Actual: The compiler cannot evaluate `sum/n` be[illegible]e `n` has no declaration, and `arr` is also unavailable to the function.
7. **Line 10 — `:` instead of `;`**
   Actual: The array declaration has invalid C syntax, so the compiler reports a syntax error and stops.
   Supposed: The declaration should correctly create an integer array containing five values.
8. **Line 11 — `average` is not declared**
   Actual: The compiler finds no function or variable named `average`, resulting in an error.
   Supposed: The program should call the `avg` function that was defined above.
9. **Line 11 — `data` is not used as a function argument correctly**
   Actual: `average data` does not follow C's function-call syntax, producing a syntax error.
   Supposed: The array `data` should be passed to the average function as an argument.
10. **Line 11 — `avg()` is never called**
    Actual: The intended average function is not invoked at all; instead, the invalid expression involving `average` is encountered.
    Supposed: The program should call the average function and obtain its returned floating-point value.
11. **Line 11 — Invalid expression `(average data, 5)`**
    Actual: The compiler cannot parse this as a valid function call, so the program does not compile.
    Supposed: The program should pass the array and its size to the averaging function.
12. **Line 11 — `%f` does not receive a valid floating-point result**
    Actual: Because the expression itself is invalid, compilation fails before `printf` executes.
    Supposed: `%f` should print the decimal average returned by the function.

Step 4

now give the corrected version of the full program add an in line comment on every line you changed stating which bug number it fixes below the code state the exact the corrected program will print

Here is the corrected version, with a comment on **every changed line** identifying the bug(s) it fixes:

```c
#include <stdio.h>

float avg(int arr[], int n) {                    // Fixes bugs 1, 4, 5, 14
    int sum = 0;                                 // Fixes bug 2

    for (int i = 0; i < n; i++) {                // Fixes bugs 3, 5, 6
        sum += arr[i];
    }

    return (float)sum / n;                       // Fixes bugs 7, 8
}

int main() {
    int data[5] = {10, 20, 30, 40, 51};        // Fixes bug 9

    printf("average =%f\n", avg(data, 5));       // Fixes bugs 10, 11, 12, 13
                                                 // and supplies data/n required by bug 14

    return 0;
}
```

Exact output

```
average =30.200001
```

Important: Mathematically, the average is **30.2**, but `%f` prints a `float` with 6 digits after the decimal point. Because the function returns `float`, the exact displayed value can be `30.200001` due to floating-point representation.

If you want the output to be exactly `30.200000`, the calculation/return type would need to be handled with higher precision.

ChatGPT can make mistakes. Check important info.

Ask anything Think

Activity A : recording sheet

Questions	Step 1	Step 2	Step 2
Number of bugs reported	14	14	14
Was the division bug caught?	Yes	Yes explained	Yes fixed
Did the ai jump ahead to the fix ?	No	No	Yes
Was any reported “bug” not a bug?	No	No	No

Activity B: structured vs unstructured

Questions	Step 1	Step 2	Step 2
Bugs correctly identified	14/14	14/14	14/14
Explanation for each bug	No	Yes	Yes
Changed line commented	No	No	Yes
Expected output stated and correct	No	No	Yes